

LegalEase AI – Phase 2: Requirement Analysis

Project: LegalEase AI Development

Phase: 02 / Requirement Analysis

Status: Approved Specification

1. INTRODUCTION

The **Requirement Analysis Phase** serves as the second crucial milestone in the architectural and system lifecycle of **LegalEase AI**. Building upon the conceptual framework established during the initial Brainstorming and Ideation Phase, this stage systematically translates high-level concepts into well-defined operational, structural, and technical prerequisites.

During this phase, all operational parameters—including user input models, legal query processing mechanics, Large Language Model (LLM) integration hooks, RESTful API pathways, document parsing requirements, software dependencies, and fundamental security protocols—are comprehensively cataloged and analyzed. The primary objective is to establish an unambiguous, technical baseline that ensures seamless execution during subsequent design and implementation phases.

2. OBJECTIVES

The strategic goals of the Requirement Analysis Phase encompass a thorough evaluation of both system behaviors and technical infrastructure. The core objectives include:

- **Functional Definition:** Formally specify core functional capabilities and user-facing interactions.
- **Technical Architecture:** Identify core technology stacks, runtime environments, and architectural frameworks.
- **Frontend & Backend Scope:** Detail user interface requirements and server-side processing workflows.
- **AI & LLM Integration:** Formulate prompt structures, API communication protocols, and response simplification logic.
- **Document Processing & Parsing:** Outline criteria for handling legal contracts, queries, and uploads.
- **Dependencies & Security:** Define required third-party libraries, environment management, and foundational data protection guidelines.
- **System Contracts:** Map clear input and output schemas across all application boundaries.

3. REQUIREMENT IDENTIFICATION

System requirements are segmented into two distinct domain categories:

Functional Requirements

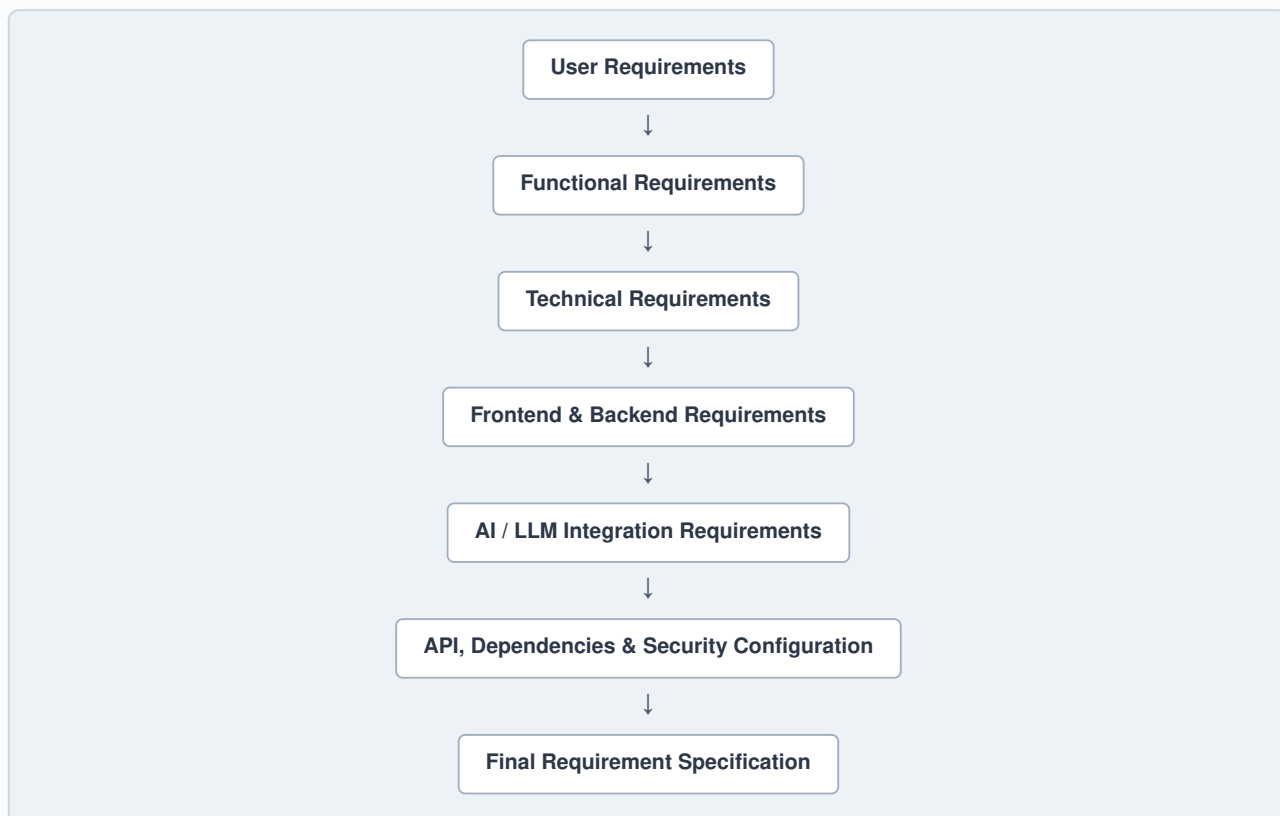
- **Query Ingestion:** The system must accept user-submitted natural language legal questions.
- **Legal Processing & Routing:** Requests must be parsed and routed for general legal query interpretation.
- **AI Orchestration:** The backend must establish secure communication with external AI/LLM models.
- **Response Simplification:** Complex legal terminologies must be converted into plain, easily understandable language.
- **Document Handling:** Support context-aware processing for legal documents and uploaded files.
- **User Interface Presentation:** A clean, responsive UI must present generated responses clearly to the user.

Technical Requirements

- **Frontend Interface:** Web-based presentation layer built with lightweight, accessible standards.
- **Backend Framework:** Python-based REST API application server (Flask).
- **AI Service Integration:** Secure REST API connections to LLM providers (e.g., OpenAI / Gemini API).
- **Configuration & Secrets:** Safe environment variable management (`python-dotenv`) for API keys.
- **Network & Connectivity:** Reliable HTTP/HTTPS connection handling for remote API execution.
- **Security Standard:** Data sanitization, input validation, and secure handling of user submissions.

4. PHASE WORKFLOW

The flow of requirements follows a sequential dependency structure from high-level user expectations down to concrete technical specifications:



5. DETAILED EXPLANATION

5.1 User Requirements

Users require a frictionless method to submit complex legal queries or plain text questions and receive rapid, simplified legal insights.

Example Scenario:

User Prompt: "What is a rental agreement?"

Expected Outcome: A concise explanation outlining lease agreements, landlord/tenant obligations, and core legal terms in accessible language.

5.2 Frontend Requirements

The user interface must remain intuitive and stripped of unnecessary visual clutter to accommodate users without technical or legal expertise. Essential elements include:

- Application Header & Title
- Multiline Text Input Area for prompts and legal documents
- Submission Action Controls (Submit / Reset)
- Dynamic Response Display Box with loading/status indicators

5.3 Backend Requirements

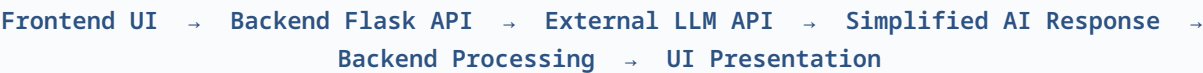
The backend acts as the bridge connecting the user-facing client with external language models. Its core duties encompass receiving payloads, performing input validation, managing API auth, handling response parsing, and managing error states.

5.4 AI / LLM Requirements

The LLM engine must accurately digest legal concepts and simplify dense legalese. *Note: The system provides informational assistance and explicit disclaimers that output does not constitute professional legal advice.*

5.5 API Requirements & Communication Flow

Communication between system modules adheres to the following synchronous flow:



6. IMPLEMENTATION CONCEPT

Below is the baseline Python / Flask backend prototype implementing endpoint validation and response handling:

```
from flask import Flask, request, jsonify app = Flask(__name__) # Endpoint to handle legal questions @app.route("/ask", methods=["POST"]) def ask_question(): data = request.get_json() if not data or "question" not in data: return jsonify({"error": "Question is required"}), 400 question = data.get("question") return jsonify({ "question": question, "message": "Question received successfully" }) if __name__ == "__main__": app.run(debug=True)
```

7. EXAMPLE INPUT AND OUTPUT

Sample Input Payload (POST /ask)	Initial Backend Validation Response
<pre>{ "question": "What is a rental agreement?"}</pre>	<pre>{ "question": "What is a rental agreement?", "message": "Question received successfully"}</pre>

8. DEPENDENCIES & CONFIGURATION

To ensure secure execution and modular deployment, configuration details and third-party dependencies are strictly segregated:

File	Purpose / Content
requirements.txt	Defines core libraries: Flask, python-dotenv, requests
.env	Stores sensitive variables: AI_API_KEY=your_api_key_here

9. EXPECTED OUTCOME & CONCLUSION

The completion of the Requirement Analysis Phase establishes a well-documented technical blueprint for **LegalEase AI**. By rigorously mapping functional capabilities, technical dependencies, API workflows, and security guardrails, this specification bridges concept ideation with formal architectural design. It serves as the primary technical baseline for the upcoming System Design and Implementation Phase.